

DEEP LEARNING

Unit – IV

Autoencoders

Autoencoders

- An **autoencoder** is a neural network that is trained to attempt to copy/reproduce its input to its output
- The network may be viewed as consisting of two parts
 - an encoder function $h = f(x)$
 - a decoder that produces a reconstruction $r = g(h)$

- **Architecture**

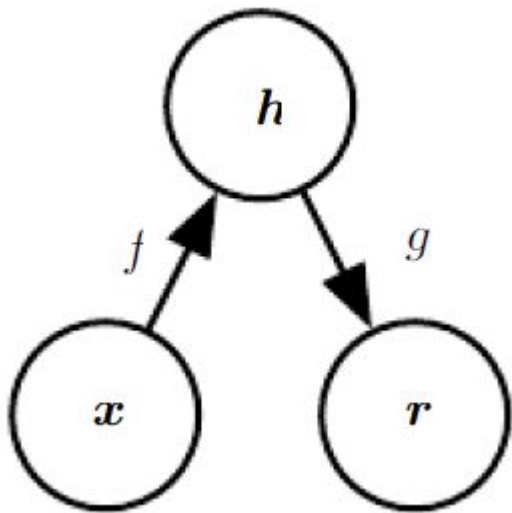


Figure 14.1: The general structure of an autoencoder, mapping an input x to an output (called reconstruction) r through an internal representation or code h . The autoencoder has two components: the encoder f (mapping x to h) and the decoder g (mapping h to r).

Applications of Autoencoders

- Dimensionality reduction
- Information retrieval
- Data Denoising
- Feature learning/Representational Learning

- **Data Denoising (Denoising Autoencoders)**
- Trained to reconstruct clean data from noisy inputs.
- Removes noise while preserving important structure.

Example: Removing noise from medical images or audio signals.

Feature Learning / Representation Learning

- Automatically learn meaningful features from raw data.
- Features can be used for classification, clustering, or regression.

Example: Learning facial features from images without manual design.

Advantages - Dimensionality reduction

- Autoencoders compress high-dimensional data into a low-dimensional code.
- Used as a nonlinear alternative to PCA.

Example: Reducing image features before classification.

- Lower-dimensional representations can improve performance on many tasks, such as classification.
- Models of smaller spaces consume **less memory and runtime**.
- The hints provided by the mapping to the lower-dimensional space **aid generalization**

Information retrieval

- **Information retrieval**, the task of finding entries in a database that resemble a query entry.
- If we train the dimensionality reduction algorithm to produce a code that is low dimensional and *binary*, then we can store all database entries in a hash table mapping binary code vectors to entries
- This hash table allows us to perform information retrieval by returning all database entries that have the same binary code as the query.
- We can also search over slightly less similar entries very efficiently, just by flipping individual bits from the encoding of the query.
- This approach to information retrieval via dimensionality reduction and binarization is called **semantic hashing** and has been applied to both textual input and images

Undercomplete Autoencoders

- At first glance, copying the input x to the output r may sound useless.
- However, we are typically not interested in the output of the decoder.
- Instead, we hope that training the autoencoder to perform the input **copying task will result in h taking on useful properties.**(**useful internal representation h .**)
- Training the network to reconstruct the input forces the hidden layer h to **capture important features** of the data.
- One way to **obtain useful features from the autoencoder** is to constrain h to have smaller dimension than x .
- *An autoencoder whose code dimension is less than the input dimension is called **undercomplete***

Undercomplete Autoencoders

- The **learning process** is described simply as **minimizing a loss function**

$$L(\mathbf{x}, g(f(\mathbf{x})))$$

- Where L is a loss function **penalizing $g(f(\mathbf{x}))$** for being dissimilar from $\mathbf{x}$, such as the mean squared error.
- When the **decoder is linear and L is the mean squared error**, an undercomplete Autoencoder learns to span the same **subspace as PCA**.
- Autoencoders with **nonlinear encoder functions f and nonlinear decoder functions g** can thus learn a more powerful nonlinear generalization of PCA.

Regularized Autoencoders

- **Undercomplete autoencoders**, with code dimension **less than the input** dimension, can learn the most salient features of the data distribution.
- We have seen that these **autoencoders fail to learn** anything useful if the encoder and decoder are given too much capacity.
- A similar problem occurs if the hidden code is allowed to have dimension equal to the input
- The **overcomplete** case in which the hidden code has dimension greater than the input.
- **In these cases**, even a linear encoder and linear decoder can learn to copy the input to the output without learning anything useful about the data distribution.

Regularized Autoencoders

- Regularized autoencoders provide the ability to train any architecture of autoencoder successfully, choosing the code dimension and the capacity of the encoder and decoder based on the complexity of distribution to be modeled
- Rather than limiting the model capacity by keeping the encoder and decoder shallow and the code size small, regularized autoencoders use a loss function that encourages the model to have other properties besides the ability to copy its input to its output.
- These other properties include
 - ☐ sparsity of the representation,
 - ☐ smallness of the derivative of the representation,
 - ☐ robustness to noise or to missing inputs.
- A regularized autoencoder can be nonlinear and overcomplete but still learn something useful about the data distribution even if the model capacity is great enough to learn a trivial identity function.

Regularized Autoencoders

- **Sparse Autoencoders**
- **Denoising Autoencoders**
- **Regularizing by Penalizing Derivatives**

Sparse Autoencoders

- A sparse autoencoder is simply an autoencoder whose training criterion involves a sparsity penalty $\Omega(h)$ on the code layer h , in addition to the reconstruction error: $L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(h)$
- where $g(h)$ is the decoder output and $h = f(x)$, the encoder output
- Sparse autoencoders are typically used to learn features for another task such as classification.
- The penalty $\Omega(h)$ simply as a regularizer term added to a feedforward network whose primary task is to copy the input to the output and possibly also perform some supervised task that depends on these sparse features.

Sparse Autoencoders

- Regularized maximum likelihood corresponds to maximizing $p(\theta \mid x)$, which is equivalent to maximizing $\log p(x \mid \theta) + \log p(\theta)$.
- The $\log p(x \mid \theta)$ term is the usual **data log-likelihood term** and the $\log p(\theta)$ term, the **log-prior over parameters**, incorporates the preference over particular values of θ .

- Assume a model with visible variables x and latent variables h , with an explicit joint distribution $p_{\text{model}}(x, h) = p_{\text{model}}(h)p_{\text{model}}(x | h)$.
- $p_{\text{model}}(h)$ is the model's prior distribution over the latent variables, representing the model's beliefs prior to seeing x .
- The log-likelihood can be decomposed as

$$\log p_{\text{model}}(\mathbf{x}) = \log \sum_h p_{\text{model}}(\mathbf{h}, \mathbf{x})$$

- Maximizing $\square$ The $\log p_{\text{model}}(h)$ term can be sparsity-inducing

$$\log p_{\text{model}}(\mathbf{h}, \mathbf{x}) = \log p_{\text{model}}(\mathbf{h}) + \log p_{\text{model}}(\mathbf{x} | \mathbf{h})$$

Denoising Autoencoders

- Rather than adding a penalty Ω to the cost function, autoencoder learns something useful **by changing the reconstruction error** term of the cost function.
- Autoencoders minimize some function $L(x, g(f(x)))$
- where L is a loss function penalizing $g(f(x))$ for being dissimilar from x , such as the L^2 norm of their difference.
- A denoising autoencoder or DAE instead minimizes $L(x, g(f(\tilde{x})))$
- where $\tilde{x}$ is a copy of x that has been corrupted by some form of noise.
- Denoising autoencoders must therefore **undo this corruption** rather than simply copying their input.

Regularizing by Penalizing Derivatives

- Another strategy for regularizing an autoencoder is to use a penalty Ω as in sparse autoencoders, $L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h}, \mathbf{x})$.
- but with a different form of Ω :
$$\Omega(\mathbf{h}, \mathbf{x}) = \lambda \sum_i ||\nabla_{\mathbf{x}} h_i||^2.$$
- This forces the model to learn a function that does not change much when $\mathbf{x}$ changes slightly.
- An autoencoder regularized in this way is called a contractive autoencoder or CAE

Contractive Autoencoders

- The contractive autoencoder introduces an explicit regularizer on the code $h = f(x)$, encouraging the derivatives of f to be as small as possible

$$\Omega(h) = \lambda \left\| \frac{\partial f(x)}{\partial x} \right\|_F^2.$$

- The penalty $\Omega(h)$ is the squared Frobenius norm (sum of squared elements) of the Jacobian matrix of partial derivatives associated with the encoder function.

Thank You

